SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	M.HARINI	**Reg.No.:**	192424011
Department:	AI & DS	**Date:**	

Code of Conduct:

I M.HARINI (192424011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.HARINI

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. REGISTER TRANSFER AND ALU OPERATIONS

AIM

To design and simulate register transfer operations and perform basic arithmetic and logical operations such as addition, subtraction, AND, and OR using multiple registers and an ALU.

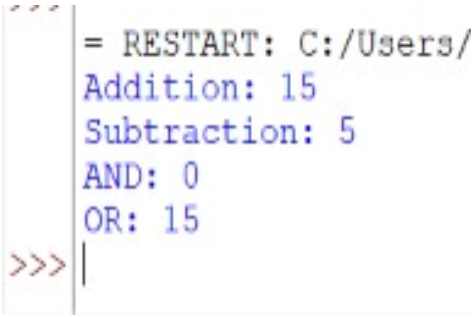
ALGORITHM

1. Initialize registers R1, R2, R3, and R4.
2. Store input values in R1 and R2.
3. Transfer the values to the ALU.
4. Perform addition and store the result in R3.
5. Perform subtraction and store the result in R4.
6. Perform AND and OR operations.
7. Display all register values and ALU results.

CODE

```
R1 = 10
R2 = 5
R3 = R1 + R2
print("Addition:", R3)
R4 = R1 - R2
print("Subtraction:", R4)
R3 = R1 & R2
print("AND:", R3)
R4 = R1 | R2
print("OR:", R4)
```

OUTPUT



RESULT

Register transfer and ALU operations were successfully simulated using Python. Addition, subtraction, AND, and OR operations produced the expected results.

2. SINGLE-BUS AND MULTI-BUS ORGANIZATION

AIM

To simulate single-bus and multi-bus computer organization using Python and compare their data transfer time and efficiency.

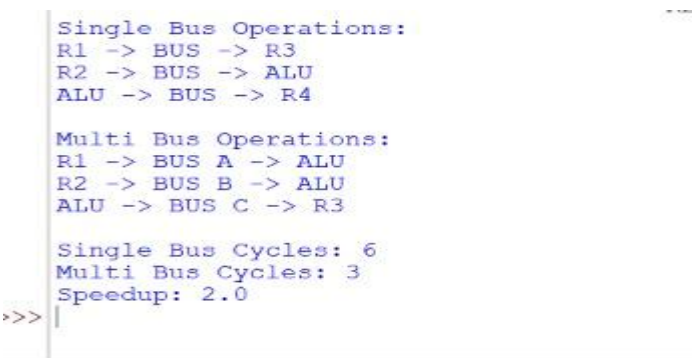
ALGORITHM

1. Initialize registers R1, R2, R3, and R4.
2. Perform data transfers using a single bus.
3. Count the number of cycles required.
4. Perform the same operations using multiple buses.
5. Count the cycles required.
6. Compare the execution time of both organizations.
7. Display the results.

CODE

```
single_bus = [
    "R1 -> BUS -> R3",
    "R2 -> BUS -> ALU",
    "ALU -> BUS -> R4"
]
multi_bus = [
    "R1 -> BUS A -> ALU",
    "R2 -> BUS B -> ALU",
    "ALU -> BUS C -> R3"
]
single_cycles = 6
multi_cycles = 3
print("Single Bus Operations:")
for x in single_bus:
    print(x)
print("\nMulti Bus Operations:")
for x in multi_bus:
    print(x)
print("\nSingle Bus Cycles:", single_cycles)
print("Multi Bus Cycles:", multi_cycles)
print("Speedup:", single_cycles / multi_cycles)
```

OUTPUT



RESULT

Single-bus and multi-bus organizations were successfully simulated. The multi-bus system required fewer cycles and provided better data-transfer efficiency.

3. FIVE-STAGE INSTRUCTION PIPELINE

AIM

To simulate a five-stage instruction pipeline using Python and compare its performance with a non-pipelined processor.

ALGORITHM

1. Define the five stages: IF, ID, EX, MEM, and WB.
2. Define a set of instructions.
3. Execute the instructions through the five pipeline stages.
4. Calculate the non-pipelined execution time.
5. Calculate the pipelined execution time.
6. Calculate speedup.
7. Display the pipeline execution sequence and results.

CODE

```
instructions = ["I1", "I2", "I3", "I4"]
stages = ["IF", "ID", "EX", "MEM", "WB"]
print("Pipeline Execution:\n")
for cycle in range(1, len(instructions) + len(stages)):
    print("Cycle", cycle, end=": ")
    for i, inst in enumerate(instructions):
        stage = cycle - i
        if 1 <= stage <= 5:
            print(inst + "-" + stages[stage-1], end="  ")
    print()
n = 10
non_pipeline = n * 5
pipeline = 5 + n - 1
speedup = non_pipeline / pipeline

print("\nNon-pipelined cycles:", non_pipeline)
print("Pipelined cycles:", pipeline)
print("Speedup:", round(speedup, 2))
```

OUTPUT

```
Pipeline Execution:

Cycle 1: I1-IF
Cycle 2: I1-ID  I2-IF
Cycle 3: I1-EX  I2-ID  I3-IF
Cycle 4: I1-MEM  I2-EX  I3-ID  I4-IF
Cycle 5: I1-WB  I2-MEM  I3-EX  I4-ID
Cycle 6: I2-WB  I3-MEM  I4-EX
Cycle 7: I3-WB  I4-MEM
Cycle 8: I4-WB

Non-pipelined cycles: 50
Pipelined cycles: 14
Speedup: 3.57
>>>
```

RESULT

The five-stage instruction pipeline was successfully simulated. The pipelined processor required fewer cycles and achieved a speedup of approximately **3.57 times** compared with the non-pipelined system.

4. PIPELINE HAZARDS

AIM

To simulate data, control, and structural hazards in a processor pipeline using Python and demonstrate hazard-resolution techniques such as stalling, data forwarding, and branch prediction.

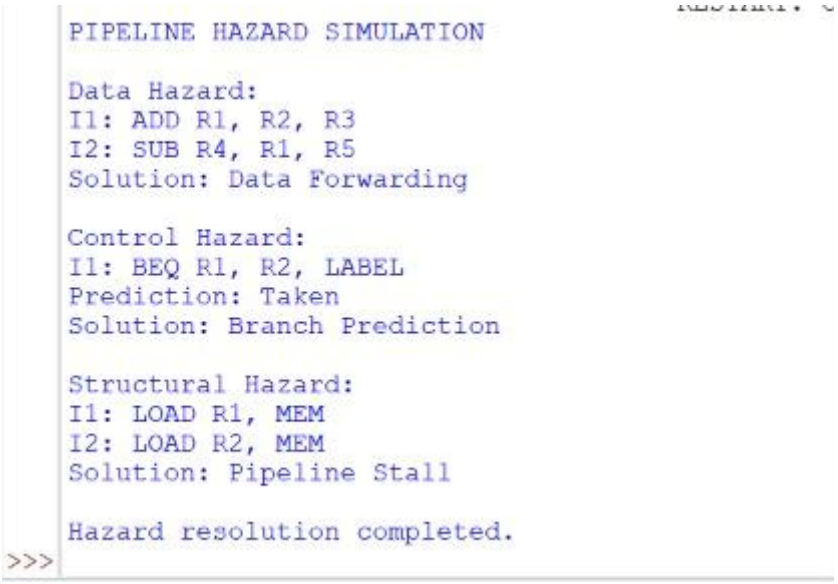
ALGORITHM

1. Define instructions that create a data hazard.
2. Identify the dependency between instructions.
3. Apply data forwarding to reduce waiting time.
4. Define a branch instruction to create a control hazard.
5. Predict the branch outcome.
6. Define instructions competing for the same resource to create a structural hazard.
7. Apply stalling to resolve the conflict.
8. Display the hazards and their solutions.

CODE

```
print("PIPELINE HAZARD SIMULATION\n")
print("Data Hazard:")
print("I1: ADD R1, R2, R3")
print("I2: SUB R4, R1, R5")
print("Solution: Data Forwarding")
print("\nControl Hazard:")
print("I1: BEQ R1, R2, LABEL")
print("Prediction: Taken")
print("Solution: Branch Prediction")
print("\nStructural Hazard:")
print("I1: LOAD R1, MEM")
print("I2: LOAD R2, MEM")
print("Solution: Pipeline Stall")
print("\nHazard resolution completed.")
```

OUTPUT



RESULT

Data, control, and structural hazards were successfully simulated. Data forwarding, branch prediction, and pipeline stalling were used to reduce the effects of pipeline hazards.

5. SUPERSCALAR, OUT-OF-ORDER, SPECULATIVE EXECUTION AND SMT

AIM

To simulate superscalar execution, out-of-order execution, speculative execution, and simultaneous multithreading (SMT) using Python and evaluate their effect on instruction throughput.

ALGORITHM

1. Define instructions belonging to two threads.
2. Allow multiple instructions to execute in the same cycle.
3. Execute independent instructions out of order when resources are available.
4. Predict branch outcomes for speculative execution.
5. Execute instructions from multiple threads using SMT.
6. Count the total number of instructions and cycles.
7. Calculate instructions per cycle (IPC).
8. Compare the result with single-issue execution.

CODE

```
thread1 = ["ADD", "SUB"]
thread2 = ["AND", "OR"]
print("SUPERSCALAR EXECUTION\n")
print("Cycle 1:", thread1[0], "+", thread2[0])
print("Cycle 2:", thread1[1], "+", thread2[1])
instructions = len(thread1) + len(thread2)
cycles = 2
ipc = instructions / cycles
print("\nTotal Instructions:", instructions)
print("Total Cycles:", cycles)
print("IPC:", ipc)
print("\nOut-of-Order: Independent instructions executed first")
print("Speculative Execution: Branch prediction used")
print("SMT: Two threads executed simultaneously")
single_issue_cycles = instructions
speedup = single_issue_cycles / cycles
print("\nSingle-Issue Cycles:", single_issue_cycles)
print("Superscalar Speedup:", speedup)
```

OUTPUT

```
SUPERSCALAR EXECUTION

Cycle 1: ADD + AND
Cycle 2: SUB + OR

Total Instructions: 4
Total Cycles: 2
IPC: 2.0

Out-of-Order: Independent instructions executed first
Speculative Execution: Branch prediction used
SMT: Two threads executed simultaneously

Single-Issue Cycles: 4
Superscalar Speedup: 2.0
>>>
```

RESULT

Superscalar, out-of-order, speculative execution, and SMT concepts were successfully simulated using Python. The processor executed multiple instructions in parallel and achieved an IPC of 2.0, resulting in a 2× speedup compared with single-issue execution.